TALENDIG · MES 4 · DATA SCIENCE PARA IA

Práctica Integradora

Pedidos de una Tienda Online — Git, Limpieza y Feature Engineering



Sistema	Python 3 + pandas + Git (VS Code)
Archivo de datos	pedidos_ecommerce.csv
Tiempo estimado	90 – 120 minutos, individual
Requisito previo	Limpieza de datos y Feature Engineering (clase del Titanic)
Objetivo	Integrar Git + limpieza + features en un pipeline propio, de principio a fin
Entrega	Repositorio en GitHub con el dataset final en main

ANTES DE EMPEZAR

Cómo usar esta práctica

Esta es una práctica integradora: vas a aplicar, en un dataset nuevo, las tres cosas que ya sabes hacer por separado — trabajar con Git de principio a fin, limpiar datos con criterio, y crear features nuevos. No es una clase en vivo: vas a tomar tú mismo cada decisión.

PASOS DE GIT

Los comandos de Git van completos — síguelos tal cual, como en la clase del Titanic.

PUNTOS DE DECISIÓN (LIMPIEZA Y FEATURES)

Antes de ver cualquier código, vas a escribir en tus propias palabras qué harías y por qué. Esto es a propósito: confirma que entendiste el criterio, no solo la sintaxis.

NOTA SOBRE TU VERSIÓN DE PANDAS

Vas a ver `pd.options.future.infer_string = False` al inicio del código. Solo cópiala tal cual; mantiene `object` en vez de `str` al revisar tipos de datos — el porqué se explica en una clase futura.

CONTEXTO

Sobre este dataset

Es un registro de pedidos de una tienda en línea, con clientes en distintos sectores de Santo Domingo. Igual que el Titanic y la práctica del gimnasio, tiene nulos, registros duplicados, y una columna numérica que en realidad es una categoría.

Columna	Significado
PedidoID	Identificador único del pedido
Cliente	Nombre completo del cliente
DireccionEnvio	Dirección de entrega, formato “Calle X #N, Sector”
CategoriaProducto	1 = Electrónica, 2 = Ropa, 3 = Hogar
CantidadArticulos	Cantidad de artículos en el pedido
PrecioUnitario	Precio por artículo, en USD
MetodoPago	Efectivo, Tarjeta o Transferencia
TiempoEntregaDias	Días que tomó la entrega
ComentarioCliente	Comentario opcional (campo reciente, casi todo nulo)

PASO 0

Crea tu propio repositorio

A diferencia de la clase del Titanic, aquí el nombre del repositorio lo eliges tú — con una sola regla: debe incluir la palabra `practica` o `talendig`, para que siga siendo identificable como parte del curso.

> **EJEMPLOS VÁLIDOS**
> `mi-practica-talendig-mes4` · `talendig-ecommerce-practica` · `practica-integradora-[tu-nombre]`

Crea el repositorio en GitHub (botón **New repository**): Visibilidad **Public**, con README, y `.gitignore` con plantilla Python.

GIT · PASO 1

Traer el repositorio a tu computadora

```
git clone https://github.com/tu-usuario/tu-repositorio.git
cd tu-repositorio
code .
```

Estructura de carpetas (igual que en la clase del Titanic):

```
mkdir notebooks
mkdir data
touch notebooks/practica_ecommerce.py
```

Coloca el archivo `pedidos_ecommerce.csv` dentro de la carpeta `data/`.

GIT · PASO 2

Crear tu rama de trabajo

```
git checkout -b feature/limpieza-features-ecommerce
git branch
```

Debe aparecer `* feature/limpieza-features-ecommerce` con el asterisco en la rama activa.

LIMPIEZA · PASO 1

Cargar y explorar el dataset

```
import pandas as pd

pd.options.future.infer_string = False

df = pd.read_csv("data/pedidos_ecommerce.csv")

print("Forma del dataset:", df.shape)
print("\nPrimeras filas:")
print(df.head())
print("\nTipos de datos:")
print(df.dtypes)
```

Salida esperada:

```
Forma del dataset: (74, 9)

Primeras filas:
   PedidoID        Cliente  ... TiempoEntregaDias  ComentarioCliente
0        38    Laura Ortiz  ...               3.0                NaN
1        42  Luis Castillo  ...               NaN                NaN
2         7      Joel Diaz  ...               4.0                NaN
3        61    Rafael Cruz  ...               1.0                NaN
4         3   Hector Mejia  ...               6.0                NaN

[5 rows x 9 columns]

Tipos de datos:
PedidoID               int64
Cliente               object
DireccionEnvio        object
CategoriaProducto      int64
CantidadArticulos      int64
PrecioUnitario       float64
MetodoPago            object
TiempoEntregaDias    float64
ComentarioCliente     object
dtype: object
```

> **SOBRE LA RUTA DEL ARCHIVO**
>
> `"data/pedidos_ecommerce.csv"` es una ruta relativa a la carpeta donde ejecutas el script — por eso es importante correrlo desde la **raíz del repositorio** (donde están las carpetas `notebooks/` y `data/` una al lado de la otra), no desde dentro de `notebooks/`. Si lo corres desde la terminal, asegúrate de estar parado en la raíz antes de ejecutar `python notebooks/practica_ecommerce.py`.

LIMPIEZA · PASO 2

Diagnosticar los valores nulos

```
print(df.isnull().sum())
print()
print((df.isnull().sum() / len(df) * 100).round(2))
```

Salida esperada:

```
PedidoID             0
Cliente              0
DireccionEnvio       0
CategoriaProducto    0
CantidadArticulos    0
PrecioUnitario       0
MetodoPago           2
TiempoEntregaDias    13
ComentarioCliente    53
dtype: int64

PedidoID             0.00
Cliente              0.00
DireccionEnvio       0.00
CategoriaProducto    0.00
CantidadArticulos    0.00
PrecioUnitario       0.00
MetodoPago           2.70
TiempoEntregaDias    17.57
ComentarioCliente    71.62
dtype: float64
```

PUNTO DE DECISIÓN

Antes de escribir código...

Tienes tres columnas con nulos: `TiempoEntregaDias` (17.57%), `MetodoPago` (2.70%) y `ComentarioCliente` (71.62%). Para cada una, responde antes de escribir cualquier código:

1. ¿Qué harías con los nulos de TiempoEntregaDias y por qué?

Pista: es numérica. ¿Mediana o promedio? Piensa en si podría haber valores atípicos en tiempos de entrega.

2. ¿Qué harías con los nulos de MetodoPago y por qué?

Pista: es una categoría con muy pocos nulos.

3. ¿Qué harías con los nulos de ComentarioCliente y por qué?

Pista: ¿tiene sentido inventar un comentario que el cliente nunca escribió?

LIMPIEZA · PASO 3

Aplicar tu estrategia

```
# TiempoEntregaDias: complete la linea con la estrategia que elegiste
df['TiempoEntregaDias'] = df['TiempoEntregaDias'].fillna( _____________ )

# MetodoPago: complete la linea con la estrategia que elegiste
df['MetodoPago'] = df['MetodoPago'].fillna( _____________ )

# ComentarioCliente: complete la linea con la estrategia que elegiste
df = _____________

print("Nulos restantes:", df.isnull().sum().sum())
```

CÓMO CONFIRMAR QUE VAS BIEN

`df.isnull().sum().sum()` debe darte `0`, y `df.shape` debe mostrar 8 columnas en vez de 9.

LIMPIEZA · PASO 4

Eliminar duplicados

```
print("Filas antes:", len(df))
df = df.drop_duplicates()
print("Filas después:", len(df))
```

AQUÍ SÍ VAS A VER UN CAMBIO REAL

A diferencia del Titanic, este dataset sí tiene filas duplicadas de verdad. Compara el número antes y después — no deberían ser iguales.

LIMPIEZA · PASO 5

Verificar tipos de datos correctos

Una columna numérica de este dataset en realidad es una categoría — igual que `Pclass` en el Titanic. (Pista: revisa el glosario — es la que tiene 3 valores fijos: 1, 2 y 3.)

```
df['_____________'] = df['_____________'].astype('category')

print(df.dtypes)
print(df.isnull().sum())
```

■ **CHECKPOINT DE LIMPIEZA**

Si todo quedó bien: 70 filas, 8 columnas, sin nulos, y la columna de categoría ya no dice `int64`.

FEATURE ENGINEERING

Las mismas 4 técnicas, en un dominio nuevo

Vas a aplicar las mismas 4 técnicas que usaste con el Titanic (combinar columnas, bandera binaria, extraer texto con regex, agrupar en categorías) — pero ojo: aquí no siempre se aplican exactamente igual. Parte del ejercicio es notar qué cambia y por qué.

FEATURE · PASO 6

Combinar columnas — MontoTotal

En el Titanic, `FamilySize` combinaba columnas **sumando**. Aquí no tiene sentido sumar cantidad de artículos con precio — ¿qué operación matemática sí tiene sentido para calcular cuánto pagó el cliente en total?

Antes de ver el código, responde:

```
df['MontoTotal'] = (df['_____________'] * df['_____________']).round(2)

print(df[['CantidadArticulos', 'PrecioUnitario', 'MontoTotal']].head())
```

FEATURE · PASO 7

Bandera binaria — PedidoGrande

En el Titanic, `IsAlone` comparaba contra un número fijo (`== 1`). Aquí vas a comparar contra un número que **calculas del propio dataset**, no que tú eliges a mano.

```
print(df['MontoTotal'].describe())

# pista: la misma medida que usaste para rellenar nulos numericos
umbral = df['MontoTotal'].____________()
df['PedidoGrande'] = (df['MontoTotal'] > umbral).astype(int)

print(df['PedidoGrande'].value_counts())
```

PREGUNTA PARA PENSAR

Si el umbral es la mediana del propio `MontoTotal`, ¿qué proporción de pedidos esperarías que queden marcados como “grandes”? Confírmalo con el resultado real.

FEATURE · PASO 8

Extraer texto con regex — Sector

`DireccionEnvio` tiene el patrón `"Calle X #N, Sector"`. En el Titanic extrajiste una sola palabra después de la coma (el título). Aquí el sector puede tener **más de una palabra** (ej. “Los Prados”), así que el patrón cambia un poco — esta vez te lo doy completo, igual que se te dio en la clase, porque es sintaxis nueva, no una decisión de criterio.

```
df['Sector'] = df['DireccionEnvio'].str.extract(r',\s*(.+)$')

print(df['Sector'].value_counts())
print()
print(df[['DireccionEnvio', 'Sector']].head())
```

EL PATRÓN, SÍMBOLO POR SÍMBOLO

`,` busca la coma literal · `\s*` son cero o más espacios después · `(.+)` es el grupo de captura: el punto significa “cualquier carácter” y el + significa “uno o más” · `$` marca el final del texto, para capturar todo lo que queda, no solo la primera palabra.

Antes de seguir: prueba quitar el `$` del final del patrón y vuelve a correr el código. ¿Cambia el resultado? Escribe en una línea qué pasó y por qué.

FEATURE · PASO 9

Agrupar en categorías — VelocidadEntrega

Igual que `AgeGroup` en el Titanic: convierte `TiempoEntregaDias` (un número continuo) en 3 categorías: Rápida, Normal, Lenta.

```
df['VelocidadEntrega'] = pd.cut(
    df['TiempoEntregaDias'],
    bins=[_____________],
    labels=['Rapida', 'Normal', 'Lenta']
)

print(df[['TiempoEntregaDias', 'VelocidadEntrega']].head(10))
print()
print(df['VelocidadEntrega'].value_counts())
```

PISTA PARA DEFINIR LOS BINS

Necesitas 4 números para definir 3 intervalos (igual que en el Titanic necesitabas 6 números para 5 intervalos). Revisa el `describe()` de `TiempoEntregaDias` del Paso 2 para elegir límites con sentido — y no olvides un último límite alto, para capturar entregas muy atípicas.

REFLEXIÓN

Mediana vs. promedio, con datos reales

Antes de seguir, calcula la media y la mediana de `TiempoEntregaDias` incluyendo los casos con 20 días o más de entrega, y luego sin ellos.

```
tiempos = df['TiempoEntregaDias']  # usa el dataset YA limpio, antes de rellenar nulos si
guardaste una copia
# o vuelve a cargar el archivo original para esta comparación

atipicos = tiempos[tiempos >= 20]
normales = tiempos[tiempos < 20]

print("Con atípicos -> Media:", round(tiempos.mean(), 2), "| Mediana:", tiempos.median())
print("Sin atípicos -> Media:", round(normales.mean(), 2), "| Mediana:", normales.median())
```

¿Cuál medida cambió más al quitar los casos atípicos? ¿Por qué tiene sentido ese resultado?

CIERRE DEL PIPELINE

Resumen final del dataset

Usa la Forma A (la que verifica contra el DataFrame real, no la que escribe los nombres a mano) — la misma que ya conoces de la guía del Titanic.

```
print("="*50)
print("RESUMEN FINAL DEL DATASET")
print("="*50)
print(f"Filas: {df.shape[0]}")
print(f"Columnas: {df.shape[1]}")
print(f"\nColumnas nuevas creadas hoy:")
print([c for c in df.columns if c in ['MontoTotal','PedidoGrande','Sector','VelocidadEntrega']])
print(f"\nValores nulos restantes: {df.isnull().sum().sum()}")
print(df.head())

df.to_csv("data/pedidos_ecommerce_limpio.csv", index=False)
print("\n■ Archivo guardado como: data/pedidos_ecommerce_limpio.csv")
```

GIT · PASO 3

Commit, merge y push

Mismo flujo de siempre, completo de principio a fin.

```
git status
git add .
git status

git commit -m "feat: limpieza y feature engineering de pedidos de e-commerce"

git checkout main
git merge feature/limpieza-features-ecommerce

git push origin main
```

Verifica en GitHub que `notebooks/practica_ecommerce.py` y `data/pedidos_ecommerce_limpio.csv` aparecen en `main`, con un commit descriptivo.

CIERRE

Autoevaluación antes de entregar

Marca cada punto antes de dar por terminada la práctica.

- [ ] df.shape final muestra (70, 8 + 4 nuevas) — sin duplicados y sin ComentarioCliente
- [ ] df.isnull().sum().sum() da 0
- [ ] CategoriaProducto aparece como category en df.dtypes, no como int64
- [ ] Las 4 columnas nuevas existen: MontoTotal, PedidoGrande, Sector, VelocidadEntrega
- [ ] Respondiste todos los puntos de decisión con tus propias palabras
- [ ] Hiciste commit, merge a main, y push — y lo verificaste en GitHub

Práctica preparada para Talendig · Mes 4 · Data Science para IA · Junio 2026